

Ultra-High Interactivity on NVIDIA GPUs? - TileRT InferenceX

在 NVIDIA GPU 上实现超高交互性？ - TileRT InferenceX

Can TileRT software on NVIDIA GPU compete with Cerebras, Groq LPU, SambaNova? Batch Size 1, Disaggregated engine, high throughput prefill engine, high interactivity decode engine

NVIDIA GPU 上的 TileRT 软件能否与 Cerebras、Groq LPU、SambaNova 相抗衡？批量大小为 1，解耦引擎，高吞吐量预填充引擎，高交互性解码引擎

BRYAN SHAN, DANIEL NISHBALL, CAM QUILICI, AND 3 OTHERS

BRYAN SHAN, DANIEL NISHBALL, CAM QUILICI 及其他 3 位作者

AUG 10, 2026 · PAID · 付费



Premium-priced “fast modes” are proving that users will pay more for lower latency and faster tokens, potentially yielding higher gross margins. Frontier AI labs such as [OpenAI are therefore evaluating purpose-built inference systems, including Cerebras and NVIDIA Groq LPUs](#) that prioritize ultra-high interactivity over maximum batched throughput. Ultra-low latency matters most in interactive workloads, including real-time assistants, and full-duplex voice. OpenAI GPT-Live, for example, can listen and speak simultaneously, making response delay immediately perceptible to the user, [described as feeling like Ironman JARVIS](#).

高价位的“快速模式”正证明，用户愿意为更低的延迟和更快的令牌生成速度支付更多费用，这有望带来更高的毛利率。因此，前沿 AI 实验室（如 OpenAI）正在评估专用推理系统，包括 Cerebras 和 NVIDIA Groq LPU，这些系统优先考虑超高交互性，而非最大批量吞吐量。超低延迟在交互式工作负载中最为关键，包括实时助手和全双工语音。例如，OpenAI GPT-Live 能够同时听和说，使得响应延迟对用户而言立即可感知，被形容为像钢铁侠的 JARVIS 一样。

GPUs perform exceptionally well at high throughput and low-to-medium interactivity, but their architecture is less suited for ultra-low-latency inference. An 8-GPU HGX B200 server provides a theoretical HBM memory bandwidth of 64 TB/s of in aggregate.

At batch size 1, GLM-5 at NVFP4 requires only approximately 21 GB of active-parameter traffic per generated token. The B200 HBM bandwidth roofline would therefore suggest up to 3,047 tokens/s/user without speculative decoding. In practice, GPUs come nowhere close to this limit.

GPU 在高吞吐量和低至中等交互性方面表现极为出色，但其架构并不适合超低延迟推理。一台 8-GPU HGX B200 服务器提供总计 64 TB/s 的理论 HBM 内存带宽。在批处理大小为 1 的情况下，GLM-5 在 NVFP4 格式下每个生成令牌仅需约 21 GB 的活动参数流量。因此，B200 的 HBM 带宽上限表明，在没有推测解码的情况下，每个用户每秒可处理多达 3,047 个令牌。然而在实践中，GPU 远未达到这一极限。

The gap comes from latency rather than bandwidth. The traditional GPU programming model launches and synchronizes many individual kernels, whose setup and teardown overhead becomes significant at ultra-high levels of interactivity. While these latency costs are less visible at conventional serving speeds, even with CUDA graphs, they dominate as token latency approaches the sub-millisecond Time Per Output Token (TPOT) range. [Furthermore, although GPU memory bandwidth increases by roughly 2-3× each generation, memory latency has not improved at all.](#)

差距源于延迟而非带宽。传统的 GPU 编程模型会启动并同步大量独立内核，在超高交互性场景下，这些内核的建立与拆除开销变得显著。虽然这些延迟成本在常规服务速度下并不明显，即便使用 CUDA 图优化，当令牌延迟接近亚毫秒级每输出令牌时间（TPOT）范围时，它们仍会占据主导地位。此外，尽管每代 GPU 内存带宽提升约 2-3 倍，但内存延迟却毫无改善。

While using alternative hardware is popular, there are ways to use GPUs to do this too. This is where [TileRT's persistent engine](#) comes in. TileRT statically compiles the entire decode graph into a single persistent kernel on NVIDIA GPUs, maximizing overlap across computation, memory loads and stores, and communication. **On the InferenceX GLM5 FP8 744B benchmark on a single B200 decode server, tileRT has been verified to reach up to 500 tokens/s/user, approximately 3× faster than GB300 NVL72 running traditional inference engines. Iso-cost per output token, TileRT can**

achieve up to 2x faster interactivity than traditional engines.

虽然使用替代硬件很流行，但仍有方法让 GPU 实现同样效果。这正是 TileRT 持久化引擎的用武之地。TileRT 将整个解码图静态编译为 NVIDIA GPU 上的单个持久化内核，最大化计算、内存读写与通信的重叠。在单块 B200 解码服务器上运行的 InferenceX GLM5 FP8 744B 基准测试中，TileRT 已验证可实现每用户每秒 500 个令牌，速度约为运行传统推理引擎的 GB300 NVL72 的 3 倍。按每输出令牌等成本计算，TileRT 的交互性可比传统引擎快 2 倍。

We thank the TileRT maintainers for collaborating on TileRT InferenceX benchmarks and also in general thankful to the vLLM community for their amazing design on the V1 connector. TileRT comes from the same community maintainer organization that built the [widely popular TileLang DSL](#).

我们感谢 TileRT 维护团队在 TileRT InferenceX 基准测试中的合作，同时也感谢 vLLM 社区在 V1 连接器上的出色设计。TileRT 与广受欢迎的 TileLang DSL 出自同一社区维护组织。

With PD disaggregation inference technique, the hyperspecialized TileRT engine handles latency-sensitive decode while throughput-optimized engines such as vLLM and SGLang continuing to serving prefill. The TileRT decode engine is already being deployed in production at [Xiaomi for MiMo V2.5 Pro UltraSpeed](#) and [ZAI with GLM 5.1 HighSpeed](#).

通过 PD 分离推理技术，超专业化的 TileRT 引擎负责处理对延迟敏感的解码任务，而 vLLM 和 SGLang 等吞吐量优化引擎则继续处理预填充任务。TileRT 解码引擎已部署在小米 MiMo V2.5 Pro UltraSpeed 和智谱 GLM 5.1 HighSpeed 的生产环境中。

In the article, we shall deep dive into the TileRT InferenceX results, what TileRT is, how it composes with the existing inference ecosystem along with the tradeoffs and challenges with TileRT.

本文将深入探讨 TileRT InferenceX 的测试结果、TileRT 的本质、它与现有推理生态系统的整合方式，以及 TileRT 的权衡与挑战。

We will also elaborate on the tradeoffs of using TileRT on standard GPUs vs. ultra low latency specialized chips like Nvidia Groq LPU, Cerebras and Sambanova, weighing in on if there is a potential for TileRT software running on GPUs to disrupt

these specialist chips' TAM. [The SemiAnalysis Accelerator Model provides quarter by quarter estimates of Nvidia LPU30, LPU40, Cerebras WSE-3 & WSE-4 shipments and much more.](#)

我们还将详细阐述在标准 GPU 上使用 TileRT 与 Nvidia Groq LPU、Cerebras 和 SambaNova 等超低延迟专用芯片之间的权衡，评估 TileRT 软件在 GPU 上运行是否有潜力颠覆这些专用芯片的总可寻址市场（TAM）。SemiAnalysis 加速器模型提供了 Nvidia LPU30、LPU40、Cerebras WSE-3 和 WSE-4 出货量等数据的逐季度估算，以及更多内容。

InferenceX

[InferenceX is our open-source, vendor-neutral, continuously updated AI inference benchmarking and research platform.](#) We measure leading models, inference frameworks, and hardware across the latency-throughput Pareto frontier, tracking how real-world inference performance and economics improve over time.

InferenceX 是我们开源、供应商中立、持续更新的 AI 推理基准测试与研究平台。我们测量领先模型、推理框架和硬件在延迟-吞吐量帕累托边界上的表现，追踪实际推理性能和经济性随时间的变化。

Thanks for reading SemiAnalysis! This post
is public so feel free to share it.

感谢阅读 SemiAnalysis! 本文为公开内容，欢迎分享。

[Our benchmark has been widely reproduced, validated and/or supported by almost every major buyer](#) of compute from [Google Cloud](#) to [Microsoft Azure](#) to [Oracle](#), to [Meta](#) and many more. Furthermore, it has the [support of the ML community including from vLLM, LMCache, SGLang, PyTorch, Huggingface](#) and the support of [major labs like OpenAI, MiniMax, ZAI, Qwen, Moonshot Kimi, etc.](#)

我们的基准测试已被从谷歌云、微软 Azure、甲骨文到 Meta 等几乎所有主要计算买家广泛复现、验证和/或支持。此外，它还获得了 ML 社区的支持，包括 vLLM、LMCache、SGLang、PyTorch、Huggingface，以及 OpenAI、MiniMax、ZAI、Qwen、Moonshot Kimi 等主要实验室的支持。

InferenceX™ Initiative Supporters

InferenceX™ initiative is supported by many major buyers of compute and prominent members of the ML community including those from MiniMax, Moonshot Kimi, Alibaba Qwen, OpenAI, Microsoft, vLLM, PyTorch Foundation, Oracle and more.



Source: [InferenceX](#) 来源: [InferenceX](#)

[Star the InferenceX GitHub repository if you find the open-source benchmark and data useful!](#) [As previously mentioned](#), Nvidia has committed to submitting verifiable Vera Rubin numbers to InferenceX. We will have Google TPUv7 results soon, and AMD has committed to MI455X UALoE72 this year too.

如果你觉得开源基准测试和数据有用，请给 InferenceX 的 GitHub 仓库点个赞。如前所述，英伟达已承诺向 InferenceX 提交可验证的 Vera Rubin 数据。我们很快会公布谷歌 TPUv7 的结果，AMD 也承诺今年会提交 MI455X UALoE72 的数据。

SemiAnalysisAI / InferenceXPublic

NotificationsFork 250Star 1.3k

<> CodeIssues 101Pull requests 97DiscussionsActionsProjectsSecurity and qualityInsights

main843 Branches0 TagsGo to fileCode

ccquil11Refresh GLM-5.2 FP8 MI325X AgentX with MTP...8d7c906 · 15 hours ago1,955 Commits

.agents/skills/debug-agentx-runs	docs(agentx): add live cluster debugging skill (...)	2 weeks ago
.claude	Simplify reusable sweep ingestion with direct s...	3 weeks ago
.github	[Klaud Cold] codeowners: own MODELS.md a...	2 days ago
benchmarks	Refresh GLM-5.2 FP8 MI325X AgentX with MT...	15 hours ago
configs	Refresh GLM-5.2 FP8 MI325X AgentX with MT...	15 hours ago
docs	Enforce deprecated model review / 强制审阅已...	last week
experimental	CollectiveX: correct documentation claims	last week
golden_al_distribution	feat(golden-al): add GLM-5.2 MTP golden AL ...	last week
runners	Refresh GLM-5.2 FP8 H200 AgentX 2P2D with...	15 hours ago

About

Open Source Continuous Inference Benchmark Research Platform — Kimi K3 2.8T, MiniMax M3, DeepSeekv4, GLM5 - GB200 NVL72 vs MI355X vs B200 vs GB300 NVL72 & soon™ TPUv6e/v7/Trainium2/3 | 开源持续推理基准研究平台 — Kimi K2.7-Code、MiniMax M3、DeepSeekv4、GLM5 - GB200 NVL72 vs MI355X vs B200 vs GB300 NVL72, 即将推出™ TPUv6e/v7/Trainium2/3

[inferencex.com/](#)

aiamdbenchmarkcudadeepseekgb200gb300glmkimillmami355xminimaxnvidiapytorchrocmsglang

Throughput vs Interactivity Curve

吞吐量与交互性曲线

Every inference system must balance two competing goals.

每个推理系统都必须在两个相互竞争的目标之间取得平衡。

- **Interactivity (tok/s/user)** measures how quickly a single user receives tokens, the inverse of time per output token (TPOT). It determines whether a response feels snappy or sluggish.

交互性 (tok/s/用户) 衡量单个用户接收令牌的速度, 即每个输出令牌时间 (TPOT) 的倒数。它决定了响应是流畅还是迟缓。

- **Throughput (tok/s/GPU)** measures how many tokens the system produces in total across all users. It largely determines the cost per token.

吞吐量 (tok/s/GPU) 衡量系统在所有用户中总共产生的令牌数量。它很大程度上决定了每个令牌的成本。

Batching increases aggregate throughput by processing more requests together, but each user typically waits longer for each token. Small batches do the opposite: they improve per-user speed while reducing the amount of useful work each GPU completes in aggregate.

批处理通过同时处理更多请求来提高总吞吐量, 但每个用户通常需要等待更长时间才能获得每个令牌。小批量则相反: 它们提高了每个用户的速度, 但减少了每个 GPU 在总体上完成的有效工作量。

InferenceX v2: NVIDIA Blackwell Vs AMD vs Hopper - Formerly InferenceMAX

InferenceX v2: NVIDIA Blackwell vs AMD vs Hopper - 原名为 InferenceMAX

DYLAN PATEL, CAM QUILICI, AND 5 OTHERS

迪伦·帕特尔、卡姆·奎利西等 5 人

• 2月17日 2月17日



Introduction 引言

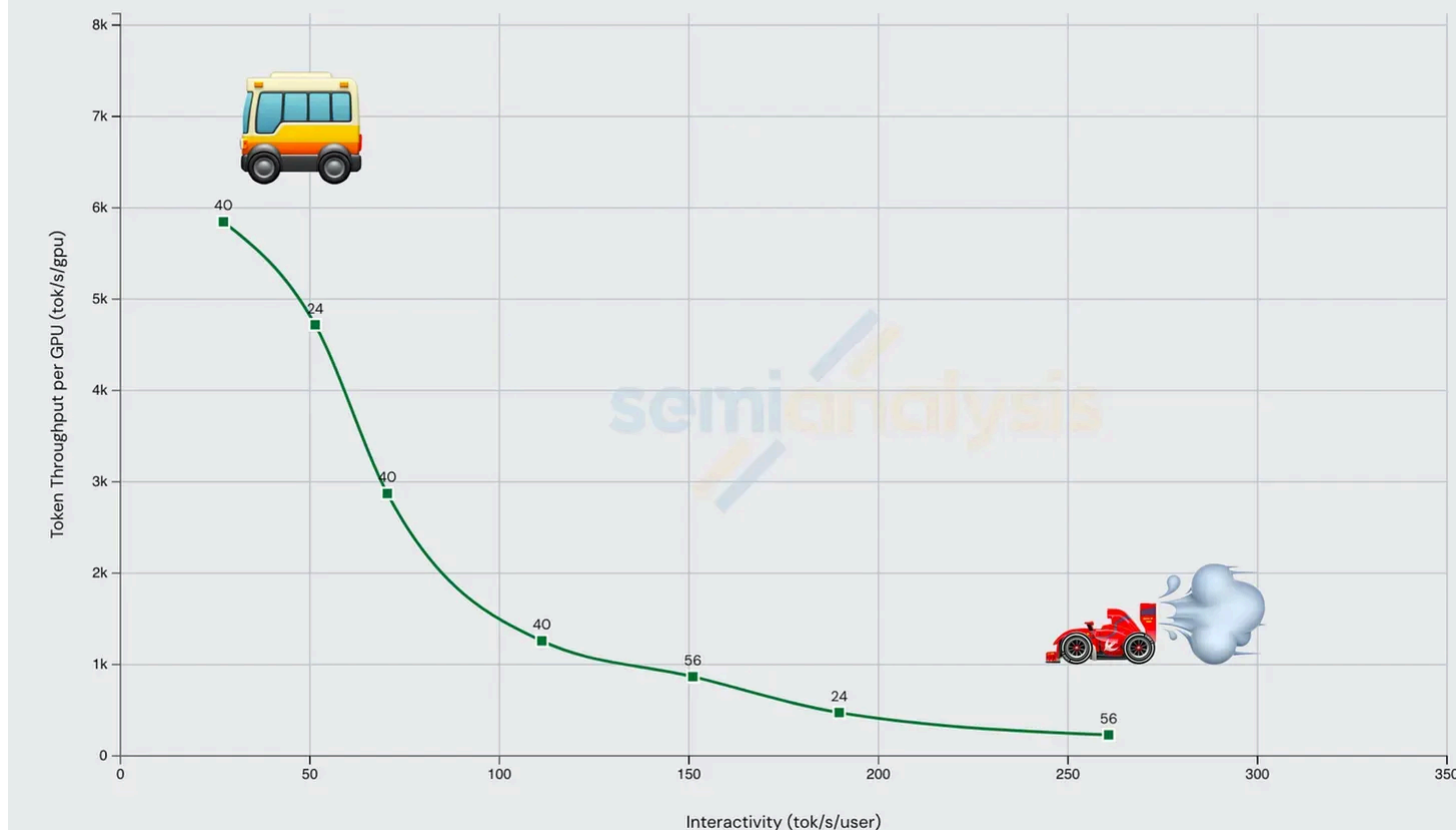
Read full story 阅读完整故事 →

A bus amortizes its cost across many passengers but makes each passenger wait for shared stops. A race car carries only one or two people and reaches the destination faster, but at much higher cost per passenger. Inference has the same trade-off: batching improves aggregate throughput and cost per token, while small batches improve per-user responsiveness. There is no one-size-fits-all operating point.

公交车将成本分摊给众多乘客，却让每位乘客在共享站点等待；赛车仅载一两人，抵达目的地更快，但每位乘客的成本却高得多。推理过程同样面临这种权衡：批处理能提升总吞吐量并降低每 token 成本，而小批量则能改善单用户响应速度。不存在放之四海而皆准的运行模式。

In the configuration shown below, increasing interactivity from roughly 25 to 260 tokens/s/user reduces per-GPU throughput from about 5,900 to 200 tokens/s/GPU. That is roughly a 30× reduction in aggregate throughput for a 10× increase in per-user speed.

在如下所示的配置中，将交互性从约 25 tokens/s/用户提升至 260 tokens/s/用户，会导致每 GPU 吞吐量从约 5,900 tokens/s/GPU 下降至 200 tokens/s/GPU。这意味着每用户速度提升 10 倍，总吞吐量却下降了约 30 倍。

Source: [SemiAnalysis](#) 来源: [SemiAnalysis](#)

TileRT results TileRT 测试结果

As we describe in the next section, GPUs already perform well in high-throughput scenarios but struggle in high-interactivity ones. This weakness has created an entire market segment for dataflow chips. TileRT targets the same weakness and therefore focuses exclusively on high-interactivity operating points.

正如我们在下一节所述，GPU 在高吞吐场景下表现优异，但在高交互性场景中却力不从心。这一短板催生了数据流芯片的整个市场。TileRT 正是瞄准这一弱点，因此专注于高交互性运行点。

SemiAnalysis is a reader-supported publication. To receive new posts and support our work, consider becoming a subscriber.

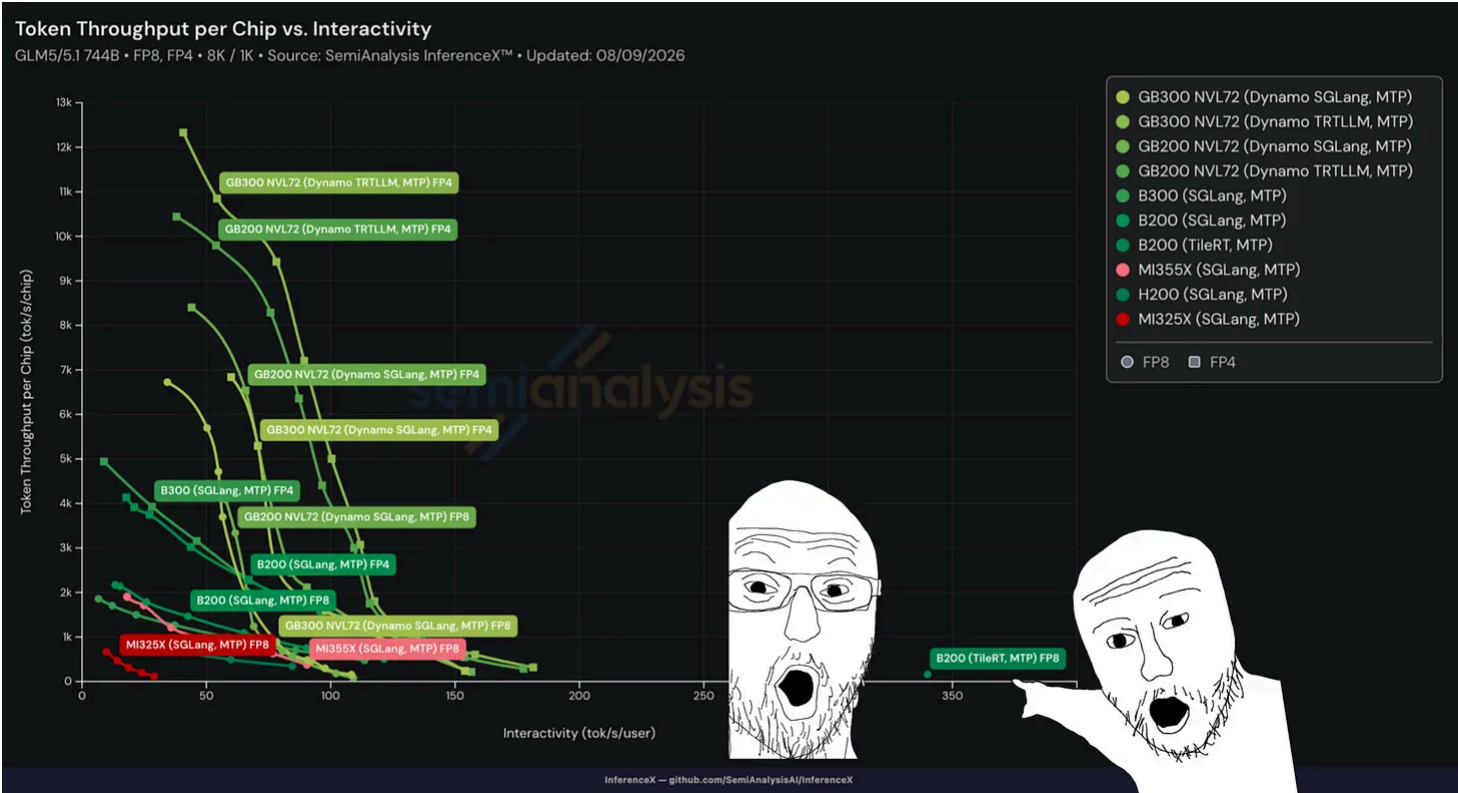
SemiAnalysis 是由读者支持的出版物。若想获取新文章并支持我们的工作，欢迎考虑订阅。

TileRT on B200 is in a class of its own. For the 8k/1k input/output token scenario, TileRT reached 340 tokens/s/user on an eight-GPU B200 node. The fastest result in the current dataset was previously 181.4 tokens/s/user on GB300 NVL72 with NVFP4 and MTP, making TileRT 1.9× faster on this metric. Of course - this is on Batch Size 1, where all that extra trouble to set up the complicated copper backplane in the case of the GB300 NVL72 does not come into play at all in boosting interactivity.

TileRT 在 B200 上的表现独树一帜。在 8k 输入/1k 输出 token 的场景下，TileRT 在八卡 B200 节点上实现了每位用户每秒 340 个 token 的处理速度。此前数据集中的最快纪录是 GB300 NVL72 采用 NVFP4 与 MTP 技术达到的 181.4 tokens/s/user，而 TileRT 在此指标上实现了 1.9 倍的性能提升。当然——这是在批处理大小为 1 的情况下，此时 GB300 NVL72 为提升交互性而搭建的复杂铜背板架构完全无法发挥作用。

Meanwhile, the fastest FP8 result was 113.6 tokens/s/user on B300 with MTP, making TileRT 3.0× faster at the same precision.

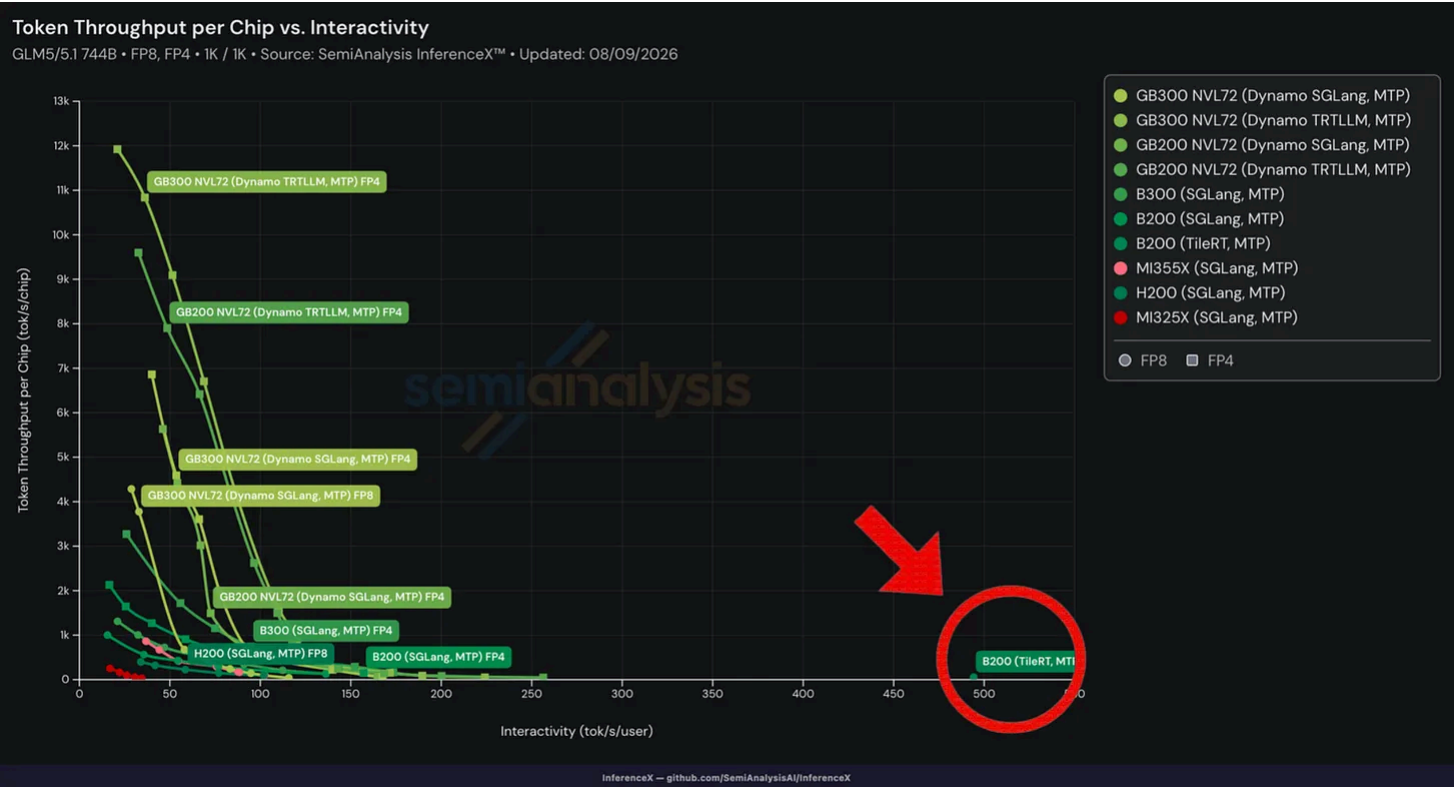
与此同时，采用 MTP 技术的 B300 在 FP8 精度下的最快结果为 113.6 tokens/s/user，而 TileRT 在相同精度下实现了 3.0 倍的性能飞跃。



B200 (TileRT, MTP) (FP8)	340.0 tok/s/user 340.0 tok/s/chip
GB300 NVL72 (Dynamo TRTLLM, MTP) (FP4)	181.4 tok/s/user 181.4 tok/s/chip

At 1k/1k input/output, TileRT FP8 reached 494.2 tokens/s/user. That was 1.9× the best conventional result, at 256.3 tokens/s/user using FP4, and 3.6× the best conventional FP8 result, at 136.3 tokens/s/user. TileRT doesn't yet have FP4 support, but it is already beating non TileRT FP4 implementations! The result is also notable because it comes from an eight-GPU B200 node rather than the 72-GPU NVLink scale-up domain of GB200 or GB300 NVL72. This comparison concerns per-user interactivity, not aggregate throughput or cost.

在 1k 输入/1k 输出 token 的场景下，TileRT FP8 达到了 494.2 tokens/s/user。这比采用 FP4 精度的最佳传统方案（256.3 tokens/s/user）快 1.9 倍，比采用 FP8 精度的最佳传统方案（136.3 tokens/s/user）快 3.6 倍。虽然 TileRT 尚未支持 FP4 精度，但其性能已超越非 TileRT 的 FP4 实现方案！更值得注意的是，这一成果来自八卡 B200 节点，而非 GB200 或 GB300 NVL72 的 72 卡 NVLink 扩展域。本对比聚焦于每位用户的交互性能，而非总吞吐量或成本。



B200 (TileRT, MTP) (FP8)	494.2 tok/s/user 494.2 tok/s/chip
GB200 NVL72 (Dynamo TRTLLM, MTP) (FP4)	256.3 tok/s/user 256.3 tok/s/chip

However - there are always tradeoffs when it comes to inference! TileRT's interactivity advantage comes with lower aggregate throughput. Conventional engines can amortize weight loads and fixed kernel costs across more users as concurrency rises. At 8K/1K input/output, the GB300 FP4+MTP point at concurrency 12 delivers approximately 240 total tokens/s/GPU while maintaining 154 tokens/s/user. TileRT delivers 160.4 total tokens/s/GPU while reaching 340 tokens/s/user.

然而，推理过程中始终存在权衡取舍！TileRT 的交互性优势是以降低总吞吐量为代价的。传统引擎能够随着并发量的增加，将权重加载和固定内核成本分摊到更多用户身上。在 8K/1K 输入/输出场景下，GB300 FP4+MTP 在并发数为 12 时，每 GPU 每秒可处理约 240 个总 token，同时保持每用户每秒 154 个 token。而 TileRT 每 GPU 每秒可处理 160.4 个总 token，同时达到每用户每秒 340 个 token。

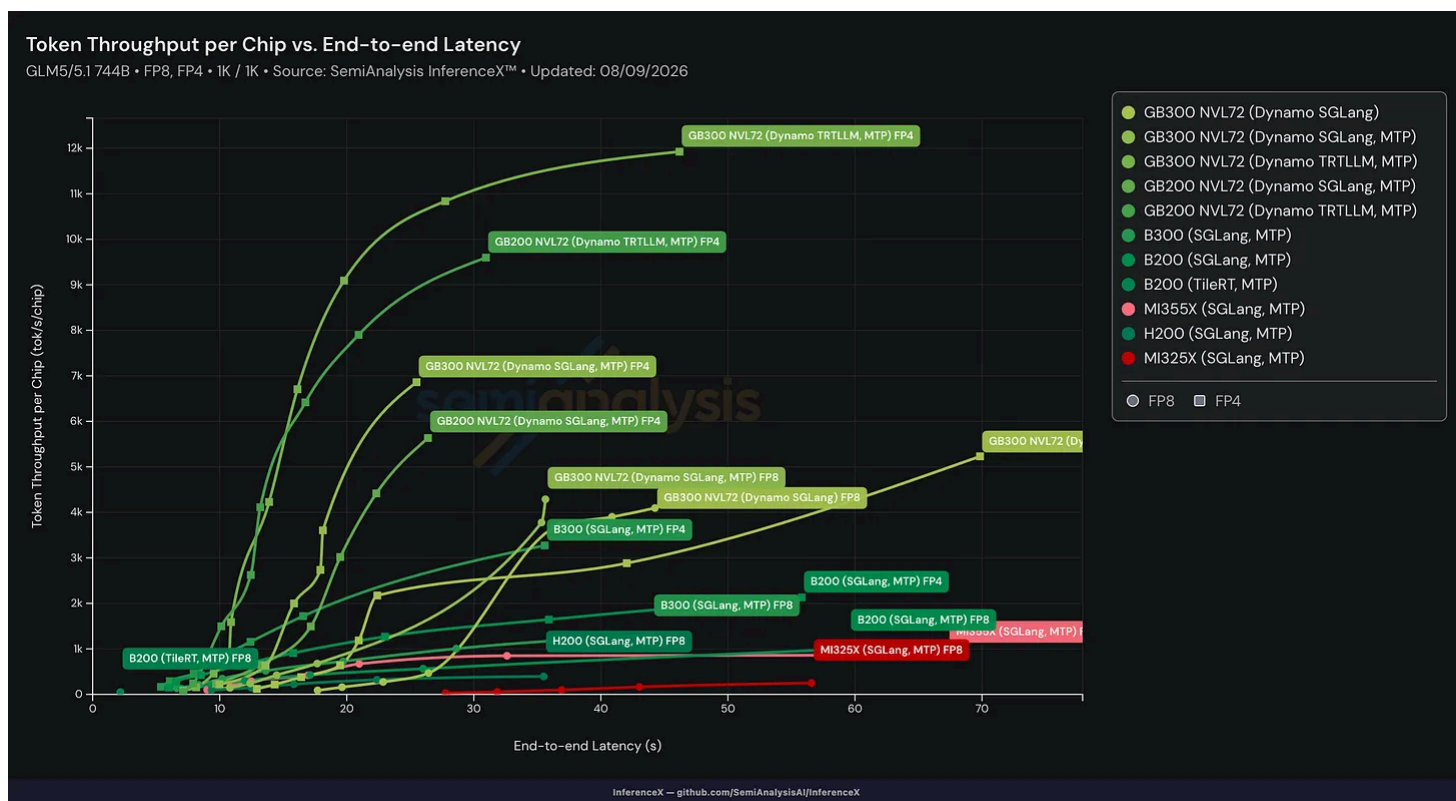
The trade-off is therefore: TileRT provides much higher per-user speed, but the conventional GB300 point completes more aggregate work per GPU. TileRT as of publication also serves only one in-flight request per decode node, making this a deliberately specialized operating point rather than a general throughput configuration. Thus, with support only for a batch size of 1 user, TileRT is not just a race car, but it is more like a private rocket ship with room for just one passenger. Engineering TileRT to support more passengers might be possible, but it is an ambitious goal.

因此，权衡在于：TileRT 提供了更高的单用户速度，但传统 GB300 方案每 GPU 能完成更多总工作量。截至本文发布，TileRT 的每个解码节点仅支持处理一个进行中的请求，这使得它成为一个刻意设计的专用运行点，而非通用的吞吐量配置。因此，由于仅支持单用户批处理，TileRT 不仅是一辆赛车，更像是一艘仅能容纳一位乘客的私人火箭飞船。要工程化实现 TileRT 支持更多乘客或许可行，但这将是一个雄心勃勃的目标。

For end-to-end latency, TileRT at FP8 outperforms the best previously recorded GLM-5.1 result by 4.5× at 1k/1k and 3.0× at 8k/1k. As expected, TileRT's time to first token (TTFT) is good but not exceptional. The decisive advantage comes from the decode tail: 3.01 seconds, compared with 6.54 seconds for the best NVFP4 + MTP competitor

and 18.18 seconds for MI355X.

对于端到端延迟，TileRT 在 FP8 精度下，在 1k/1k 和 8k/1k 场景中分别比此前最佳的 GLM-5.1 结果快 4.5 倍和 3.0 倍。不出所料，TileRT 的首词延迟（TTFT）表现良好但并非卓越。其决定性优势来自解码尾部：3.01 秒，而最佳 NVFP4+MTP 竞品为 6.54 秒，MI355X 则为 18.18 秒。



Source: InferenceX 来源: InferenceX

But What Exactly is TileRT?

但 TileRT 究竟是什么？

We briefly introduced what TileRT does and showed some benchmark results, but let's pause and explain more deeply what TileRT is and how it works. Traditional serving engines run as thousands of separate GPU program kernels launched one after another. All that setup and teardown means the GPU spends a surprising amount of time waiting, and while this setup/teardown time might not matter for low to medium interactivity inference but it definitely does for ultra high interactivity inference (aka low latency inference). Worse, each kernel writes its half-finished work out to HBM. At small batch sizes, this is a bigger problem as kernels aren't large enough to

amortize launch latency, synchronization, and scheduling overhead.

我们简要介绍了 TileRT 的功能并展示了一些基准测试结果，但让我们暂停一下，更深入地解释 TileRT 是什么以及它是如何工作的。传统推理引擎以数千个独立 GPU 程序内核的形式运行，这些内核依次启动。所有启动和关闭操作意味着 GPU 会花费大量时间等待，而这种启动/关闭时间对于中低交互性推理可能无关紧要，但对于超高交互性推理（即低延迟推理）则至关重要。更糟糕的是，每个内核都会将其未完成的工作写入 HBM。在小批量场景下，这个问题更为严重，因为内核规模不足以分摊启动延迟、同步和调度开销。

SemiAnalysis is a reader-supported publication. To receive new posts and support our work, consider becoming a subscriber.

SemiAnalysis 是一份由读者支持的刊物。若希望接收新文章并支持我们的工作，不妨考虑成为订阅用户。

[As mentioned earlier, when running TileRT at batch size 1](#), for just an single HGX H200 server (38.4TB/s of aggregate HBM memory bandwidth), the active parameter memory bandwidth stands at 42GB per token at MXFP8. In theory, if we were only bound by memory bandwidth, then even without spec decoding, inference should be able to reach up to 1,000 tok/s/user interactivity. This is obviously not the case in the real world! The roadblock is that GPUs' programming and architecture model is traditionally not built for low latency. [Even though memory bandwidth per GPU increases 2-3x each generation, memory latency has not improved at all, even as HBM prices continually increase!](#)

如前所述，当 TileRT 以批处理大小为 1 运行时，仅需一台 HGX H200 服务器（聚合 HBM 内存带宽达 38.4TB/s），每个 token 在 MXFP8 精度下所需的活跃参数内存带宽为 42GB。理论上，若仅受内存带宽限制，即便不采用推测解码，推理速度也能达到每用户每秒 1000 个 token 的交互水平。然而现实世界显然并非如此！瓶颈在于：GPU 的编程与架构模型传统上并非为低延迟而设计。尽管每代 GPU 的内存带宽提升 2-3 倍，但内存延迟却毫无改善——即便 HBM 价格持续攀升！

NVIDIA HBM Bandwidth Per Generation



Source: [SemiAnalysis](#), [Nvidia](#)

来源: [SemiAnalysis](#), 英伟达

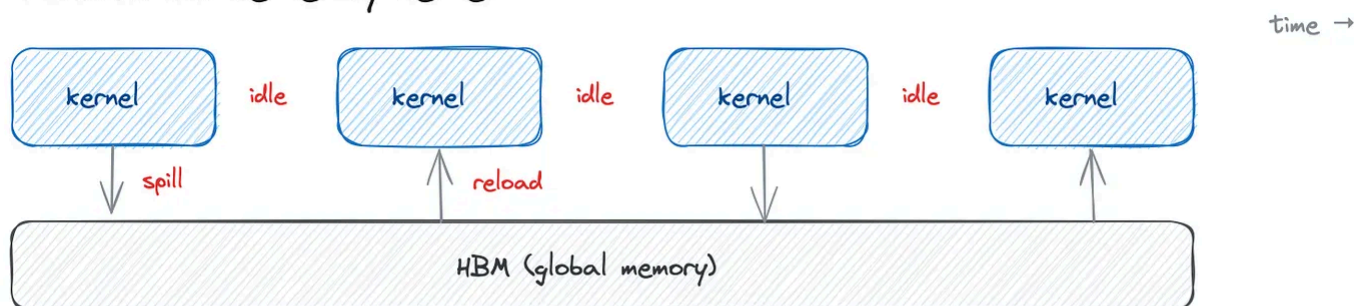
Instead of continuously launching kernels, TileRT has the GPU continuously execute a persistent pipeline, statically compiling the whole model ahead of time into a persistent Engine Kernel: the host launches once, execution stays resident on the GPU for the whole decode lifecycle, and most runtime orchestration moves into compile time.

TileRT 并非持续启动内核，而是让 GPU 持续执行持久化流水线，将整个模型提前静态编译为持久化引擎内核：主机仅启动一次，执行过程在整个解码生命周期内驻留于 GPU，且大部分运行时编排工作转移至编译阶段完成。

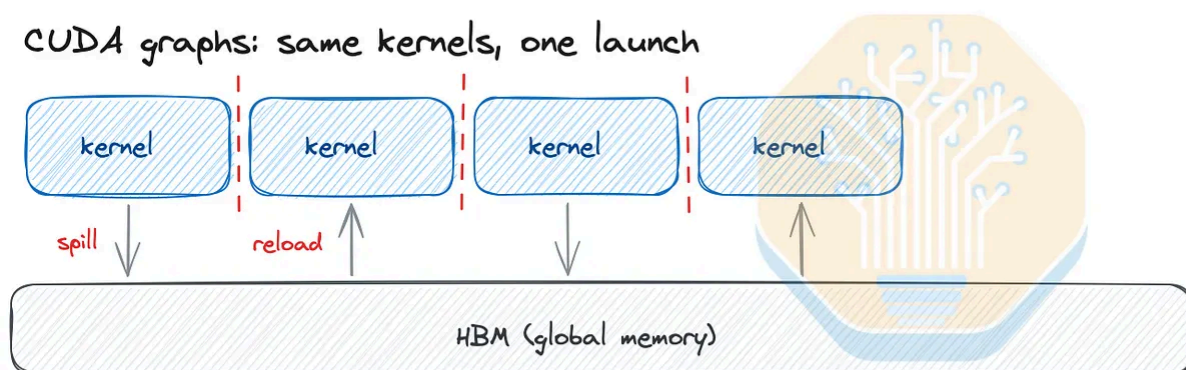
This is different from CUDA graphs, which captures the DAGs(directed acyclic graphs) of kernel launches and memcpys once, then replays it with a single `cudaGraphLaunch`. But the kernels themselves are still separate kernels, this boundary between kernels carries device-side costs and the on-chip state is wiped at every boundary. A CUDA graph optimizes the launching of kernels, while TileRT abolishes the kernel as the unit of execution.

这与 CUDA 图不同，后者会捕获内核启动和内存拷贝的有向无环图（DAG），然后通过一次`cudaGraphLaunch`进行重放。但内核本身仍然是独立的内核，这种内核之间的边界会带来设备端的开销，且片上状态在每个边界处都会被清除。CUDA 图优化了内核的启动过程，而 TileRT 则废除了将内核作为执行单元的方式。

Traditional: kernel by kernel

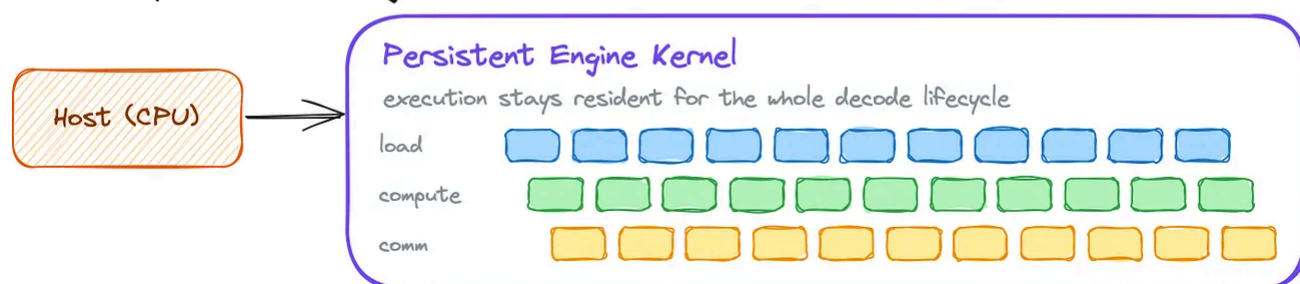


CUDA graphs: same kernels, one launch



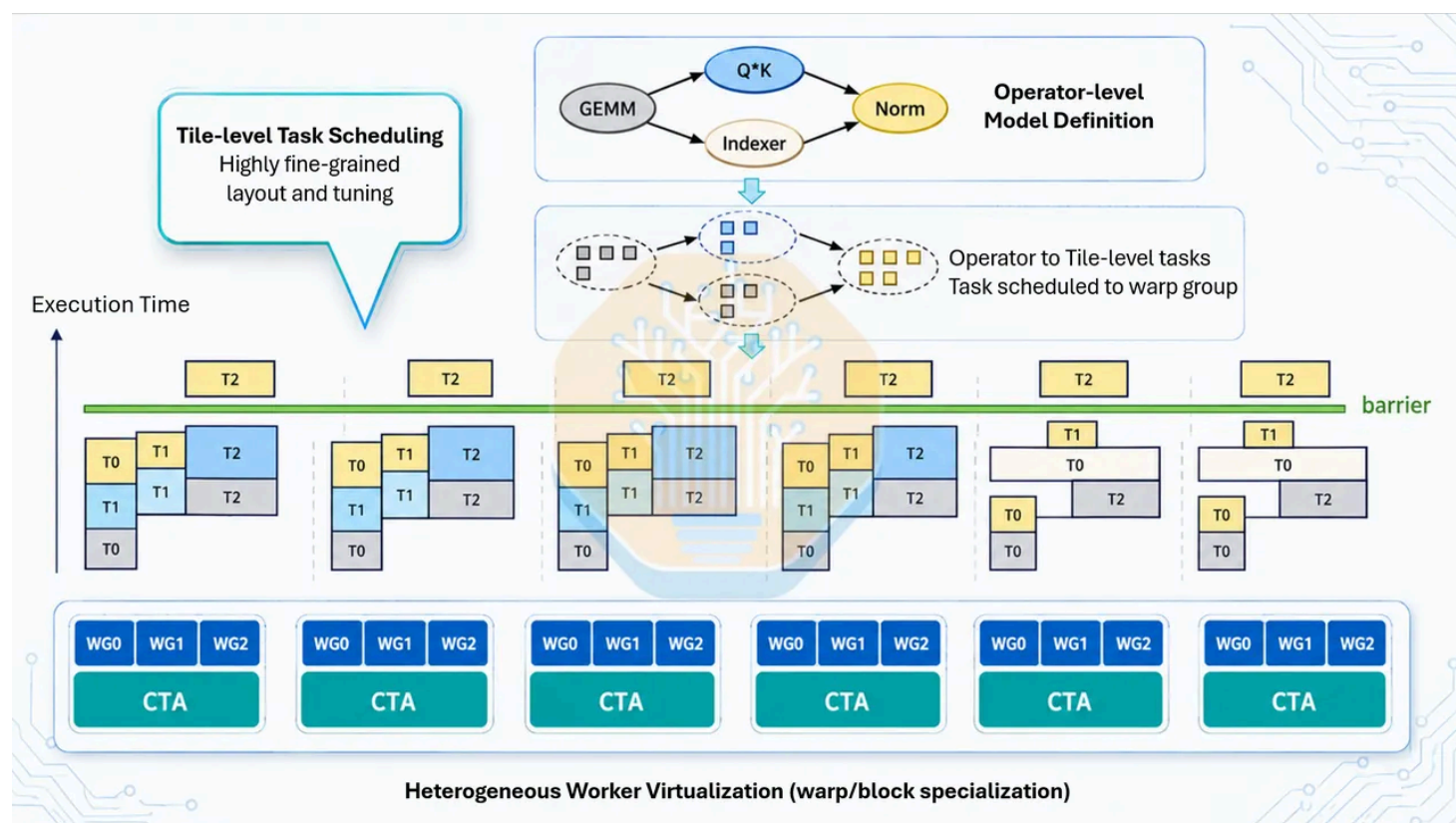
TileRT: persistent Engine Kernel

Statically compiled AoT



Also, through decomposing work into tile-level tasks with warp and block specialization, the runtime dynamically reschedules computation, I/O, and communication in a highly overlapped way. Inside the Engine Kernel, different warp groups take on different jobs: asynchronous data movement, tensor computation, and communication overlap. Where stages used to run serially as load → barrier → compute → barrier, they now overlap at tile granularity, and intermediate results flow forward through registers, shared memory, and L2 instead of repeatedly spilling to global memory. Effectively, each CTA(Cooperative Thread Array) becomes a small heterogeneous factory rather than a uniform SIMT(Single Instruction Multiple Threads) worker.

此外，通过将工作分解为瓦片级任务，并采用线程束和线程块特化，运行时系统以高度重叠的方式动态重新调度计算、输入输出和通信。在引擎内核内部，不同的线程束组承担不同的工作：异步数据移动、张量计算和通信重叠。过去各阶段以“加载→屏障→计算→屏障”的顺序串行执行，而现在它们以瓦片粒度重叠运行，中间结果通过寄存器、共享内存和 L2 缓存向前传递，而不再反复溢出到全局内存。实际上，每个协作线程数组（CTA）变成了一个小型异构工厂，而非统一的单指令多线程（SIMT）工作单元。

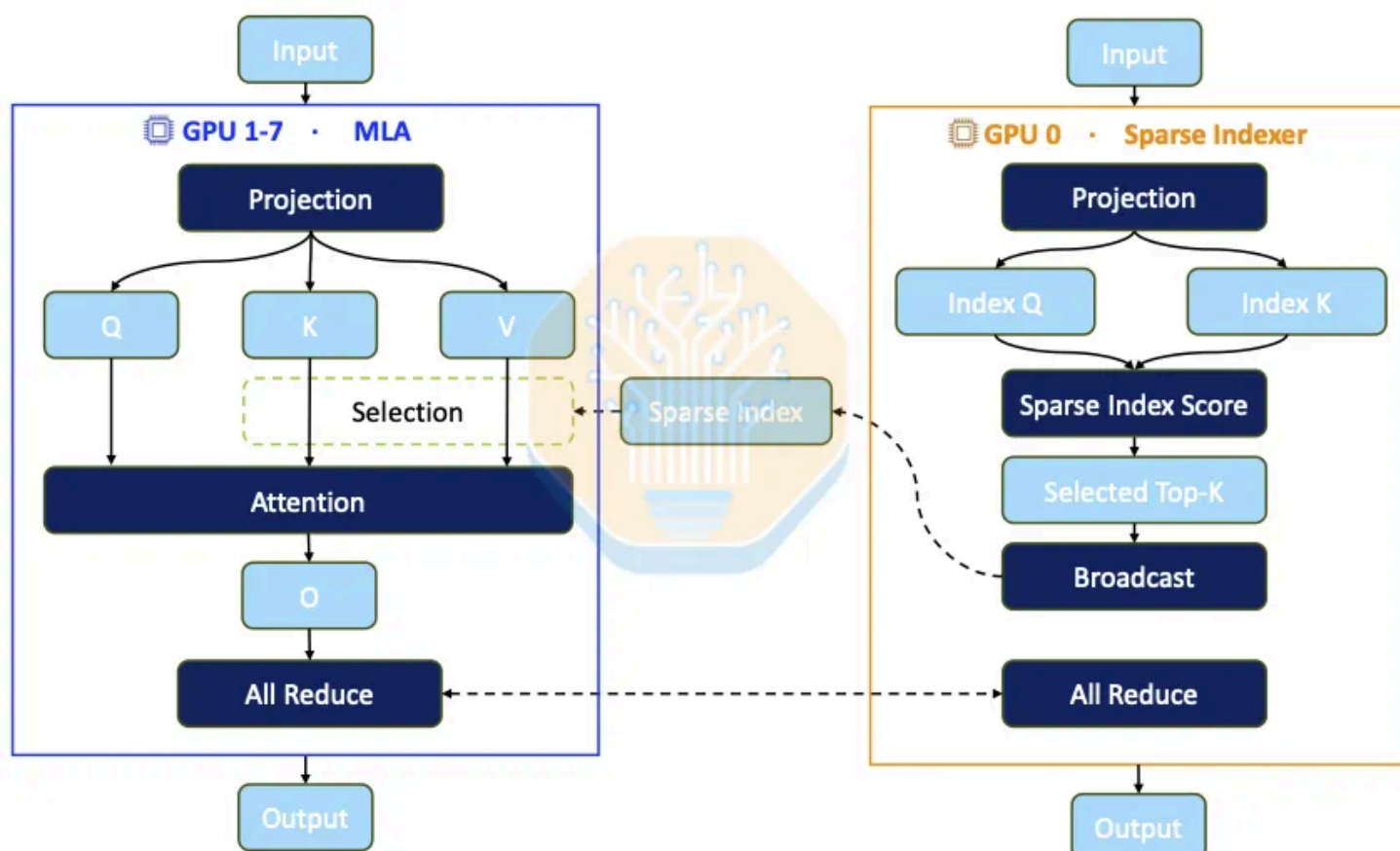


Source: [TileRT](#) 来源: [TileRT](#)

The next optimization TileRT introduces is specialization extended to whole GPUs. Most TP frameworks assume all ranks execute identical logic synchronously, but

sparse routing, Top-K selection, dynamic indexing, long-context attention, and MTP don't fit homogeneous scale-out well; they're not compute-heavy but depend on global information, so forcing every rank through them adds redundant work and synchronization amplification. So, if warps can specialize, so can GPUs. In GLM-5.1's attention layer, GPU 0 becomes a Sparse Indexer worker handling Top-K selection, sparse index construction, and routing, while GPUs 1 through 7 run the MLA workers doing RMSNorm, GEMM, flash sparse attention, and AllReduce.

TileRT 引入的下一个优化是将专业化扩展到整个 GPU。大多数张量并行（TP）框架假设所有 rank 同步执行相同的逻辑，但稀疏路由、Top-K 选择、动态索引、长上下文注意力以及 MTP 并不适合同质化的横向扩展；它们并非计算密集型，而是依赖于全局信息，因此强制每个 rank 都执行这些操作会带来冗余工作和同步放大。因此，既然 warp 可以专业化，GPU 也可以。在 GLM-5.1 的注意力层中，GPU 0 成为稀疏索引工作节点，负责处理 Top-K 选择、稀疏索引构建和路由，而 GPU 1 到 7 则运行 MLA 工作节点，执行 RMSNorm、GEMM、闪存稀疏注意力和 AllReduce。



Source: [TileRT](#) 来源: TileRT

Finally, instead of treating communication as an external stage, broadcasts, reductions, and synchronization execute directly inside the tile-level flow; with TileRT, an entire attention layer corresponds to a single kernel launch at the host, and execution shifts from compute $\rightarrow$ sync $\rightarrow$ compute toward a continuously overlapping compute $\leftrightarrow$

communication $\leftrightarrow$ compute pipeline.

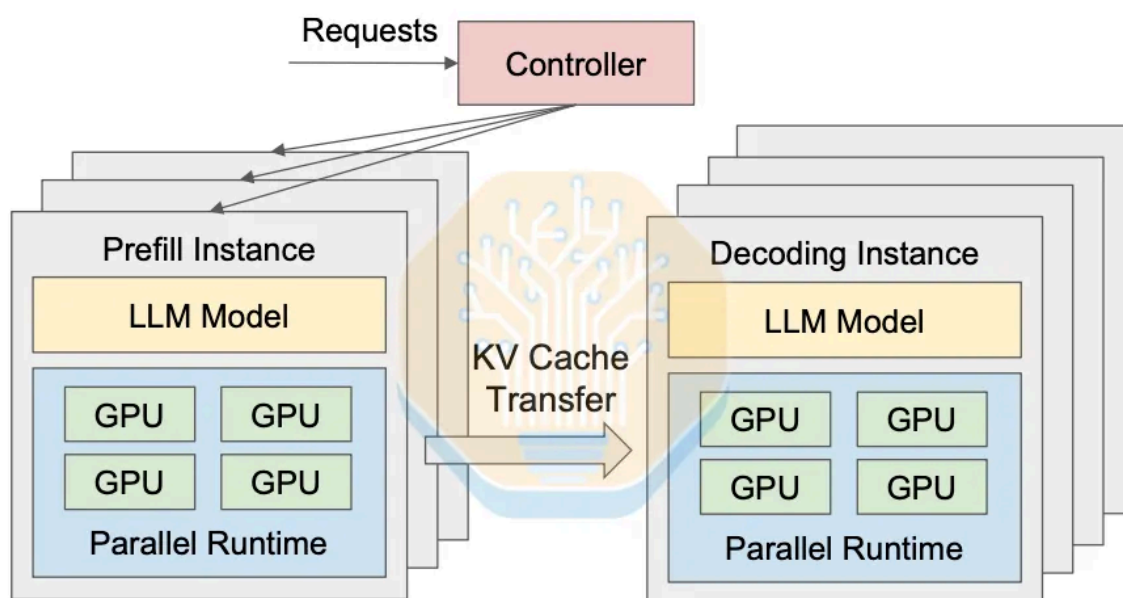
最后，TileRT 不再将通信视为外部阶段，而是将广播、归约和同步直接执行在 tile 级流程内部；使用 TileRT，整个注意力层对应主机上的单次内核启动，执行模式从“计算 $\rightarrow$ 同步 $\rightarrow$ 计算”转变为持续重叠的“计算 $\leftrightarrow$ 通信 $\leftrightarrow$ 计算”流水线。

PD Disaggregated Engine with vLLM <> TileRT

基于 vLLM <> TileRT 的 PD 分离引擎

LLM inference consists of two distinct phases: prefill and decode. Prefill processes the input prompt in parallel and is primarily compute-intensive, making aggregate throughput the key performance metric. Decode generates tokens sequentially and repeatedly accesses the growing KV cache, making it memory-intensive and highly sensitive to per-token latency.

LLM 推理包含两个不同的阶段：预填充和解码。预填充阶段并行处理输入提示，主要受计算能力限制，因此总吞吐量是关键性能指标。解码阶段则顺序生成令牌，并反复访问不断增长的 KV 缓存，这使得它受内存限制，并且对每个令牌的延迟高度敏感。



TileRT does not replace vLLM, vLLM remains the high-throughput prefill engine and the surrounding serving layer, including its scheduler, chunked prefill, prefix caching, OpenAI-compatible API, and operational tooling. Only latency-critical decode traffic moves to TileRT. TileRT is engineered to be a single-passenger rocket ship, and vLLM remains the plane, car, bus, and train.

TileRT 并不会取代 vLLM，vLLM 仍然是高吞吐量的预填充引擎以及周边的服务层，包括其调度器、分块预填充、前缀缓存、兼容 OpenAI 的 API 和运维工具。只有对延迟要求苛刻的解码流量才会转移到 TileRT。TileRT 被设计成一艘单乘客的火箭飞船，而 vLLM 则继续扮演飞机、汽车、巴士和火车的角色。

The prefill and decode phases can be disaggregated into separate nodes. With disagg, one shared vLLM prefill pool can feed two entirely different decode pools.

预填充和解码阶段可以分解到不同的节点上。通过解耦，一个共享的 vLLM 预填充池可以为两个完全不同的解码池提供数据。

- Pool A: Ultra high interactivity decode with TileRT

池 A：使用 TileRT 实现超高交互性解码

- Latency-critical requests pass through the TileRT PD Router, which instructs vLLM to generate the first token and marks the request with the destination TileRT node in kv_transfer_params.

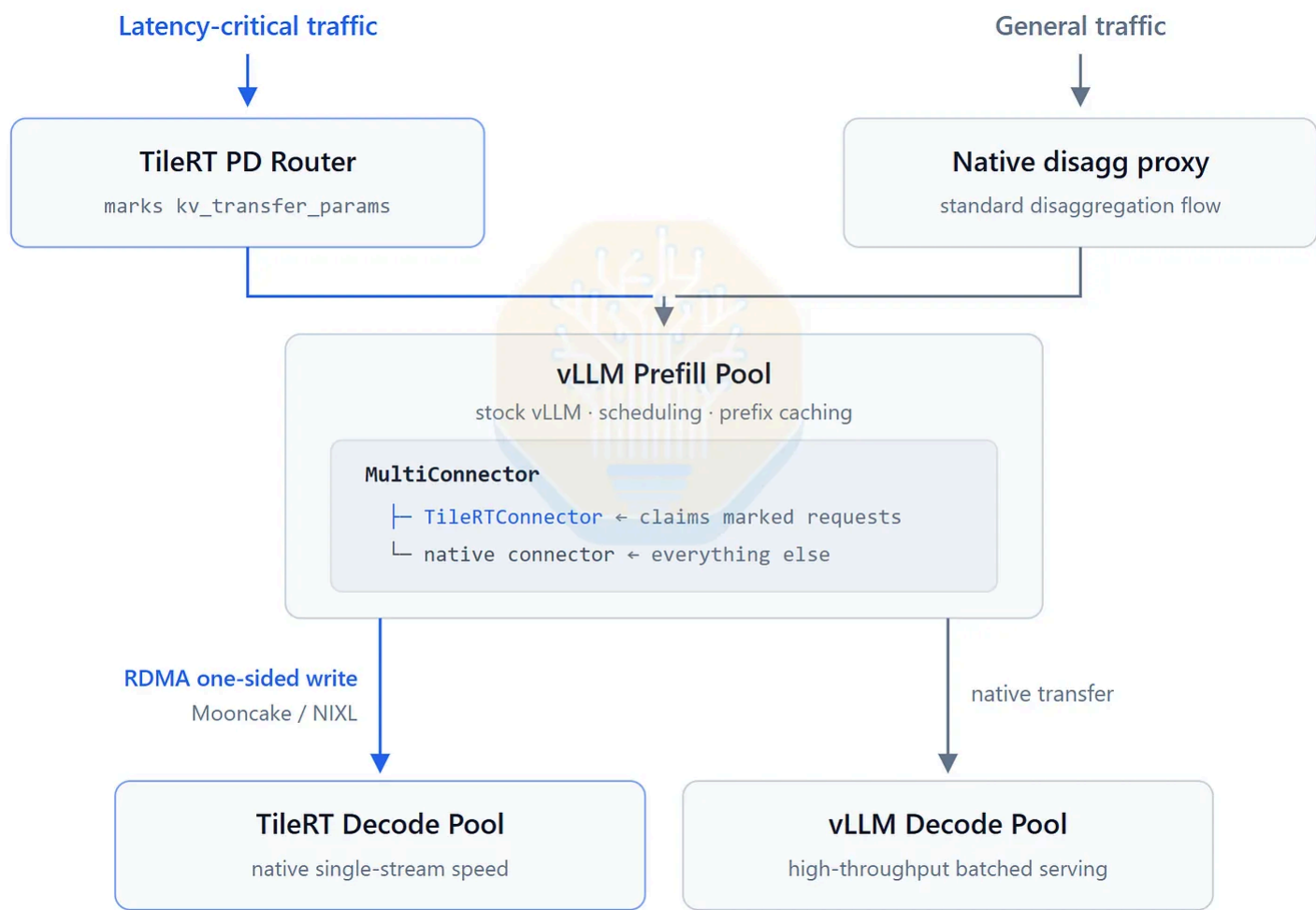
对延迟敏感的请求会经过 TileRT PD 路由器，该路由器指示 vLLM 生成第一个令牌，并在 kv_transfer_params 中标记请求的目标 TileRT 节点。

- Pool B: General low to medium interactivity decode with vLLM decode

池 B：使用 vLLM 解码实现通用低到中等交互性

- General traffic continues through vLLM's native disaggregation proxy to a conventional vLLM decode pool.

常规流量继续通过 vLLM 的原生分离代理，流向传统的 vLLM 解码池。



Source: [vLLM & TileRT](#) 来源: [vLLM & TileRT](#)

This is done via vLLM’s MultiConnector API that composes the TileRTConnector with its native connector. The TileRT connector claims only marked high interactivity traffic class requests and becomes a no-op for everything else, meaning both traffic classes can share the same prefill server. Between the Prefill and Decode, TileRT uses Mooncake Transfer Engine and NIXL Transfer Engine to move KVCache. In TileRT v0.1.5, each decode node serves one in-flight request at a time. The router gates dispatch and applies back-pressure when the node is occupied.

这是通过 vLLM 的 MultiConnector API 实现的，该 API 将 TileRTConnector 与其原生连接器组合在一起。TileRT 连接器仅处理标记为高交互性流量类别的请求，对其他请求则无操作，这意味着两种流量类别可以共享同一个预填充服务器。在预填充和解码之间，TileRT 使用 Mooncake 传输引擎和 NIXL 传输引擎来移动 KVCache。在 TileRT v0.1.5 版本中，每个解码节点一次只处理一个正在进行的请求。当节点被占用时，路由器会控制调度并施加背压。

Thanks for reading SemiAnalysis! This post
is public so feel free to share it.

How does TileRT compare to Cerebras/Groq/SambaNova?

TileRT 与 Cerebras/Groq/SambaNova 相比如何?

Purpose-built inference vendors identified the same execution bottleneck years ago, but encoded more of the solution in hardware. [The SemiAnalysis Accelerator Model has our quarter by quarter estimates of NVIDIA LPU30, LPU40, Cerebras WSE-3 & WSE-4 shipments.](#)

专用推理供应商多年前就发现了相同的执行瓶颈，但将更多解决方案编码到了硬件中。SemiAnalysis 加速器模型包含我们对 NVIDIA LPU30、LPU40、Cerebras WSE-3 和 WSE-4 出货量的逐季度估算。

Groq uses deterministic, compiler-orchestrated execution and a large on-chip SRAM hierarchy. Cerebras maps computation spatially across a wafer-scale processor; the CS-3 provides approximately 900,000 cores, 44 GB of on-chip SRAM, and 21 PB/s of memory bandwidth. SambaNova maps model graphs onto reconfigurable dataflow units backed by a tiered SRAM, HBM, and DDR memory system.

Groq 采用确定性、编译器编排的执行方式以及大型片上 SRAM 层级结构。Cerebras 将计算空间映射到晶圆级处理器上；CS-3 提供约 90 万个核心、44 GB 片上 SRAM 和 21 PB/s 的内存带宽。SambaNova 将模型图映射到可重构的数据流单元上，这些单元由分层的 SRAM、HBM 和 DDR 内存系统支撑。

SemiAnalysis is a reader-supported publication. To receive new posts and support our work, consider becoming

SemiAnalysis 是一份由读者支持的出版物。如需接收新文章并支持我们的工作，请考虑成为订阅者。

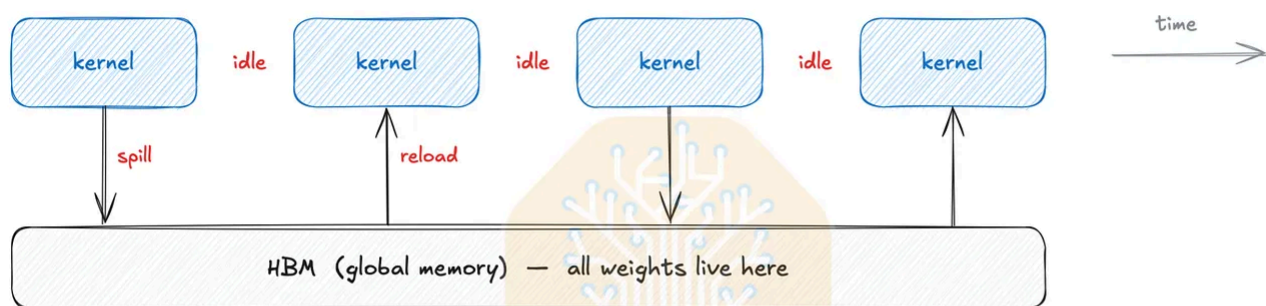
The silicon differs, but the systems share the same idea: latency-sensitive inference benefits from reducing runtime scheduling, operator boundaries, synchronization, and unnecessary movement through external memory. At large batch sizes, those costs are easier to amortize. At batch size 1, they occupy a much larger share of each token's latency.

芯片设计虽各有不同，但这些系统共享同一理念：延迟敏感的推理任务受益于减少运行时调度、算子边界、同步操作以及对外部内存的不必要访问。在大批量处理时，这些成本更容易被分摊。而在单批次处理时，它们占据了每个令牌延迟的更大比例。

GPU: kernel by kernel

every weight refetched from HBM — every layer, every token

one set of compute units is reused for every layer of the model



Dataflow silicon

activations flow through — the weights never move

Cerebras WSE · Groq LPU · SambaNova RDU



Source: SemiAnalysis 来源: SemiAnalysis

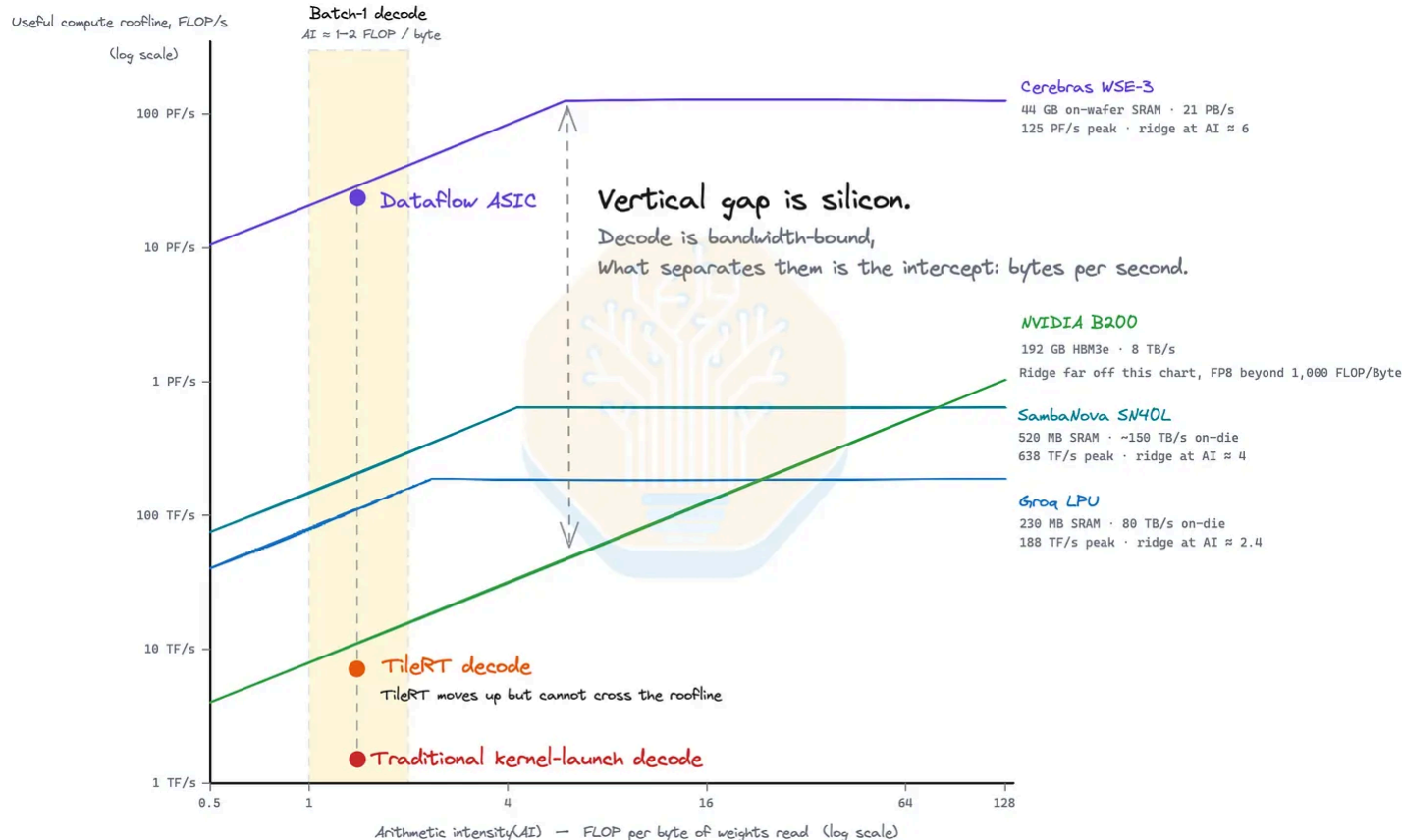
TileRT imports software analogues of several dataflow ideas: AoT scheduling, persistent execution, specialized workers, and tighter overlap between communication and computation. The resemblance is architectural rather than literal. TileRT still runs on a SIMT GPU with dynamic hardware scheduling, HBM, and a model-specific

compiled schedule.

TileRT 引入了多种数据流理念的软件对应方案：提前编译调度、持久化执行、专用工作单元，以及通信与计算之间更紧密的重叠。这种相似性体现在架构层面而非字面意义上。TileRT 仍然运行在具有动态硬件调度、HBM 和模型特定编译计划的 SIMT GPU 上。

However, TileRT is still software only: dataflow is imposed on a machine that was never specialized designed for it. A GPU carries dynamic warp schedulers, a SIMT model, and an HBM hierarchy, and TileRT gets its numbers by spending enormous compiler effort convincing that machinery to impersonate a spatial pipeline through statically expanded persistent kernels, hand-carved warp specialization, and per-model compilation against pinned driver stacks. Native dataflow silicon never fights its own substrate. Purpose-built accelerators encode more of the execution model in hardware and can avoid some of the overhead TileRT must hide in software. Their advantage still depends on the model, precision, memory hierarchy, compiler quality, system scale, and serving configuration. That is why Cerebras serves a dense 70B at speeds no eight-GPU node can reach regardless of scheduling: software can approach the HBM roofline, but it cannot raise it.

然而，TileRT 本质上仍是纯软件方案：它在一台从未为此专门设计的机器上强行施加数据流架构。GPU 配备动态线程束调度器、SIMT 模型和 HBM 层级结构，而 TileRT 通过投入巨大的编译器工程——包括静态展开的持久化内核、手工定制的线程束特化、以及针对固定驱动栈的逐模型编译——来迫使这套硬件模拟空间流水线，从而获得其性能数据。原生数据流芯片则无需与自身底层架构对抗。专用加速器将更多执行模型编码到硬件中，能够规避 TileRT 必须通过软件隐藏的部分开销。但它们的优势仍取决于模型、精度、内存层级、编译器质量、系统规模及服务配置。这正是为何 Cerebras 能以任何八 GPU 节点（无论采用何种调度策略）都无法企及的速度运行稠密 70B 模型：软件可以逼近 HBM 理论峰值，却无法提升这一上限。



Source: SemiAnalysis 来源: SemiAnalysis

The market's early answer is that purity is negotiable. TileRT's decode engine is already in production behind Xiaomi's MiMo V2.5 Pro UltraSpeed and Z.ai's GLM-5.1 HighSpeed, and the deployment pattern is the tell. Neither company procured a new dataflow chip. They carved a speed tier out of the accelerator cluster they already ran, with vLLM keeping prefill, scheduling, and the API while TileRT takes over decode behind the same endpoint. Good enough on hardware you already own tends to beat architecturally pure on hardware you have to buy.

市场早期的答案是：纯度可以协商。TileRT 的解码引擎已投入生产，服务于小米 MiMo V2.5 Pro UltraSpeed 和智谱 GLM-5.1 HighSpeed，其部署模式揭示了关键信息。这两家公司均未采购新的数据流芯片，而是在现有加速器集群中划分出高速层级——vLLM 负责预填充、调度和 API 接口，而 TileRT 则在相同端点后接管解码任务。在已有硬件上实现"足够好"的表现，往往胜过在必须购买的硬件上追求架构的纯粹性。

That points at the deeper structural problem: fungibility and flexibility in prefill-decode (PD) ratio.

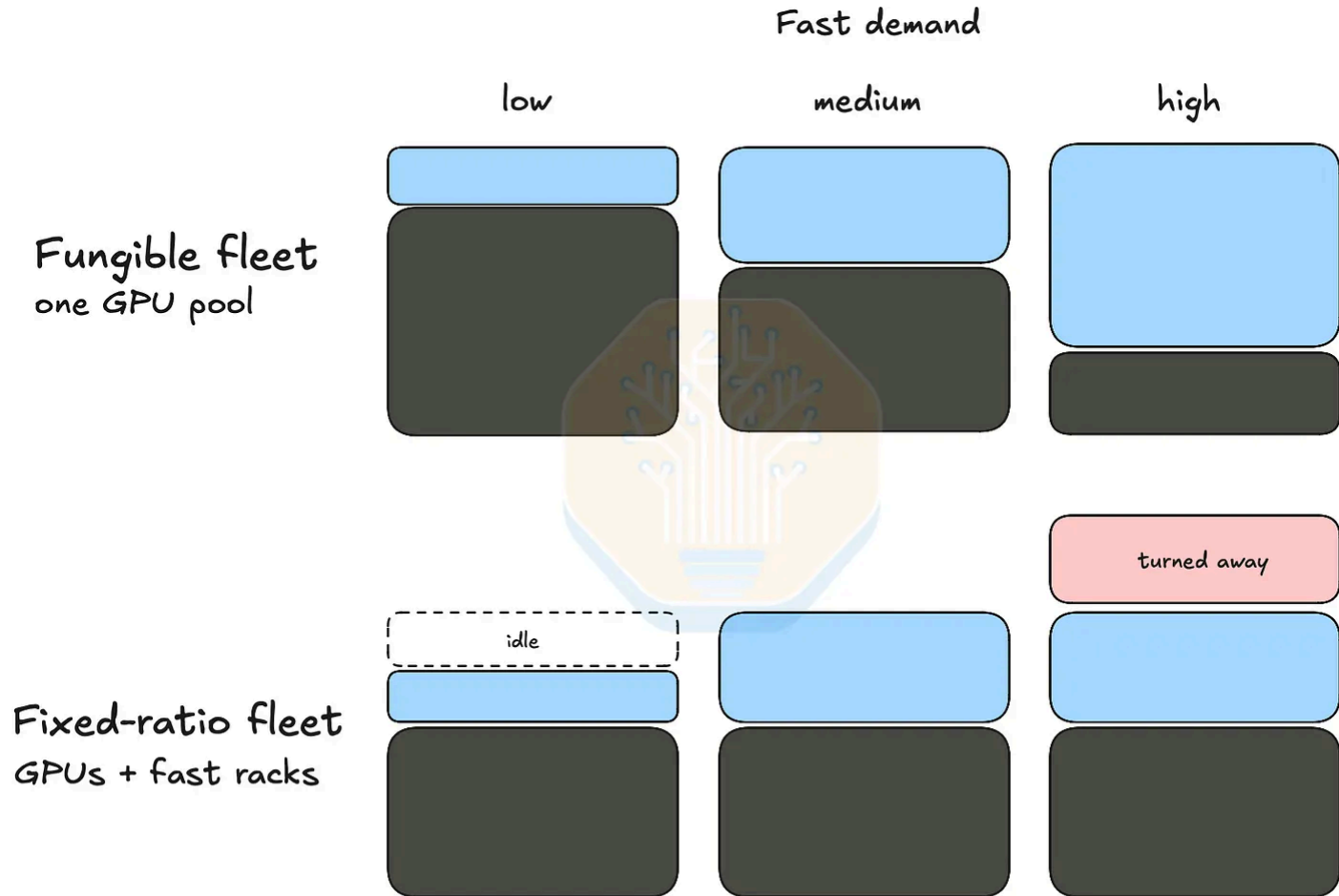
这指向了更深层次的结构性问题：预填充-解码（PD）比率的可互换性与灵活性。

A GPU pool is one liquid resource, excellent at prefill, excellent at high to medium-batch decode, [and now somewhat credibly strong at ultra-interactive decode](#), with capacity moving between those roles as software scheduler decision that can follow demand hour by hour. An ASIC fleet is the opposite: the ratio of speed-tier capacity to everything else is fixed in hardware the day the purchase order is signed. Changing the ratio of the physical fleet will take months to physically re-rack and re-cable. That would be fine if the workload mix were stable and known. Unfortunately, the split between users who need ordinary conversational latency and users, increasingly agents, who will pay for extreme-interactivity SLOs has a lot of different variables at play when estimating. Guess wrong with GPUs and you rebalance in software. Guess wrong with dedicated silicon and you either strand capital in idle speed machines or turn away the exact premium traffic you bought them for. On top of that - requirements may shift over time, so a correct guess will only be right for a limited period of time.

GPU 池是一种统一的液态资源，在预填充阶段表现出色，在中高批量解码环节表现优异，如今在超交互式解码领域也展现出相当可信的实力，其容量可在不同角色间灵活切换——只需通过软件调度决策，便能按小时需求动态调整。而 ASIC 集群则截然相反：从签署采购订单的那一刻起，各速度层级容量与其他资源的比例便由硬件固化。若要改变物理集群的配比，需要耗费数月时间进行物理机架重组和线缆重连。若 workload 组合稳定且可预测，这倒也无妨。但遗憾的是，在估算需要普通对话延迟的用户与愿意为极致交互 SLO 付费的用户（尤其是日益增多的智能体）之间的比例时，存在诸多变量。若对 GPU 的预判失误，可通过软件重新平衡；若对专用芯片的预判失误，则要么让资本闲置在空闲的高速设备中，要么将本应为其购买的优质流量拒之门外。更甚者，需求会随时间推移而变化，因此即便预判正确，其有效性也仅能维持有限的时间。

Going back to the shared prefill pool mentioned earlier, providers do not need to pay the TileRT premium for all traffic. General requests can stay on throughput-optimized vLLM or SGLang decode pools, while only latency-critical requests are routed to the TileRT decode pool.

回到之前提到的共享预填充池，服务提供商无需为所有流量支付 TileRT 的溢价。常规请求可以继续使用吞吐量优化的 vLLM 或 SGLang 解码池，只有对延迟敏感的请求才会被路由到 TileRT 解码池。



Source: SemiAnalysis 来源: SemiAnalysis

None of this kills the top of the speed market. The SRAM roofline is still better, certain sizes of models still favor it, and some workloads will always want maximum tokens per second at any price. But TileRT reframes what most buyers need: not a speed machine, but a speed tier, provisioned dynamically out of the fleet they were going to own anyway. Cerebras, Groq, and SambaNova are no longer competing against a clumsy kernel-launcher. They are competing against their own execution model, running on fungible hardware, reallocated by a config file. TileRT may be a single-passenger rocketship, but it allows providers to strap solid rocket boosters to your Metro Bus instead of having to design an entirely new launch vehicle.

这并未完全扼杀速度市场的顶端。SRAM 的极限性能依然更优，某些特定规模的模型仍更青睐它，且部分工作负载始终追求不惜代价的最大每秒令牌数。但 TileRT 重新定义了大多数买家的需求：他们需要的不是一台速度机器，而是一个速度层级，能够从他们本就计划拥有的算力集群中动态调配。Cerebras、Groq 和 SambaNova 不再是与笨拙的内核启动器竞争。他们是在与自己的执行模型竞争——这个模型运行在可互换的硬件上，仅需一个配置文件即可重新分配。TileRT 或许是一艘单人火箭飞船，但它允许服务提供商为你的城市巴士捆绑固体火箭助推器，而无需设计全新的运载工具。

Why Is TileRT Development Slow?

为什么 TileRT 开发进展缓慢？

GLM5.1 is a generation behind, and has already been deprecated on mainline InferenceX. TileRT's model catalog is very limited, currently supporting GLM-5/5.1, DeepSeek-V3.2. MiMo-V2.5-Pro-UltraSpeed is the result of a co-design partnership and has yet to be open-sourced.

GLM5.1 已落后一代，且在主线 InferenceX 上已被弃用。TileRT 的模型目录非常有限，目前仅支持 GLM-5/5.1 和 DeepSeek-V3.2。MiMo-V2.5-Pro-UltraSpeed 是协同设计合作的成果，尚未开源。

TileRT inherits ASIC vendors' biggest weakness. Static ahead-of-time compilation means a tiny model catalog (currently GLM-5/5.1 and DeepSeek-V3.2), hard-pinned dependencies, and real engineering effort per new architecture. There is no fully generic path, a persistent engine kernel means the model is statically expanded ahead of time into one resident program, so decisions have to be made on tile shapes, pipeline depth, buffer residency across registers/shared memory/L2, how warp groups split between loading, compute, and communication, where collectives get fused into the tile flow, and which GPUs take specialized roles like GLM-5.1's dedicated sparse indexer rank. Change the attention mechanism or the routing scheme and much of that schedule is invalidated. Dataflow chips also face this same issue, good compilers can be notoriously difficult to create.

TileRT 继承了 ASIC 厂商的最大弱点。静态提前编译意味着模型目录极小（目前仅支持 GLM-5/5.1 和 DeepSeek-V3.2）、依赖项高度锁定，且每适配一种新架构都需要投入大量工程工作。不存在完全通用的路径——持久化引擎内核意味着模型会被提前静态展开为一个驻留程序，因此必须对以下方面做出决策：分块形状、流水线深度、寄存器/共享内存/L2 间的缓冲区驻留策略、加载/计算/通信的 warp 组划分方式、集合通信在分块流程中的融合位置，以及各 GPU 承担的专业角色（如 GLM-5.1 专用的稀疏索引器 rank）。一旦改变注意力机制或路由方案，大部分调度计划就会失效。数据流芯片同样面临此问题——优秀的编译器往往极难构建。

Work is being done to simplify this, especially as software development can be accelerated with AI. [TileOPs](#) is intended to reduce this burden. Each operator is declared in a machine-readable manifest specifying its signature, workloads, and

roofline model. The manifest drives code generation, testing, and benchmarking against hardware bounds rather than only against earlier implementations.

相关工作正在简化这一过程，尤其是在人工智能可以加速软件开发的情况下。TileOPs 旨在减轻这一负担。每个算子都在一个机器可读的清单中声明，该清单指定了其签名、工作负载和屋顶线模型。该清单驱动代码生成、测试和基准测试，以硬件边界为基准，而不仅仅是与早期实现进行比较。

AI coding agents accelerate tuning within known templates, but novel transformations still require expert judgment. A monolithic persistent kernel also reduces the usefulness of conventional per-kernel profiler timelines, making automated feedback loops more difficult.

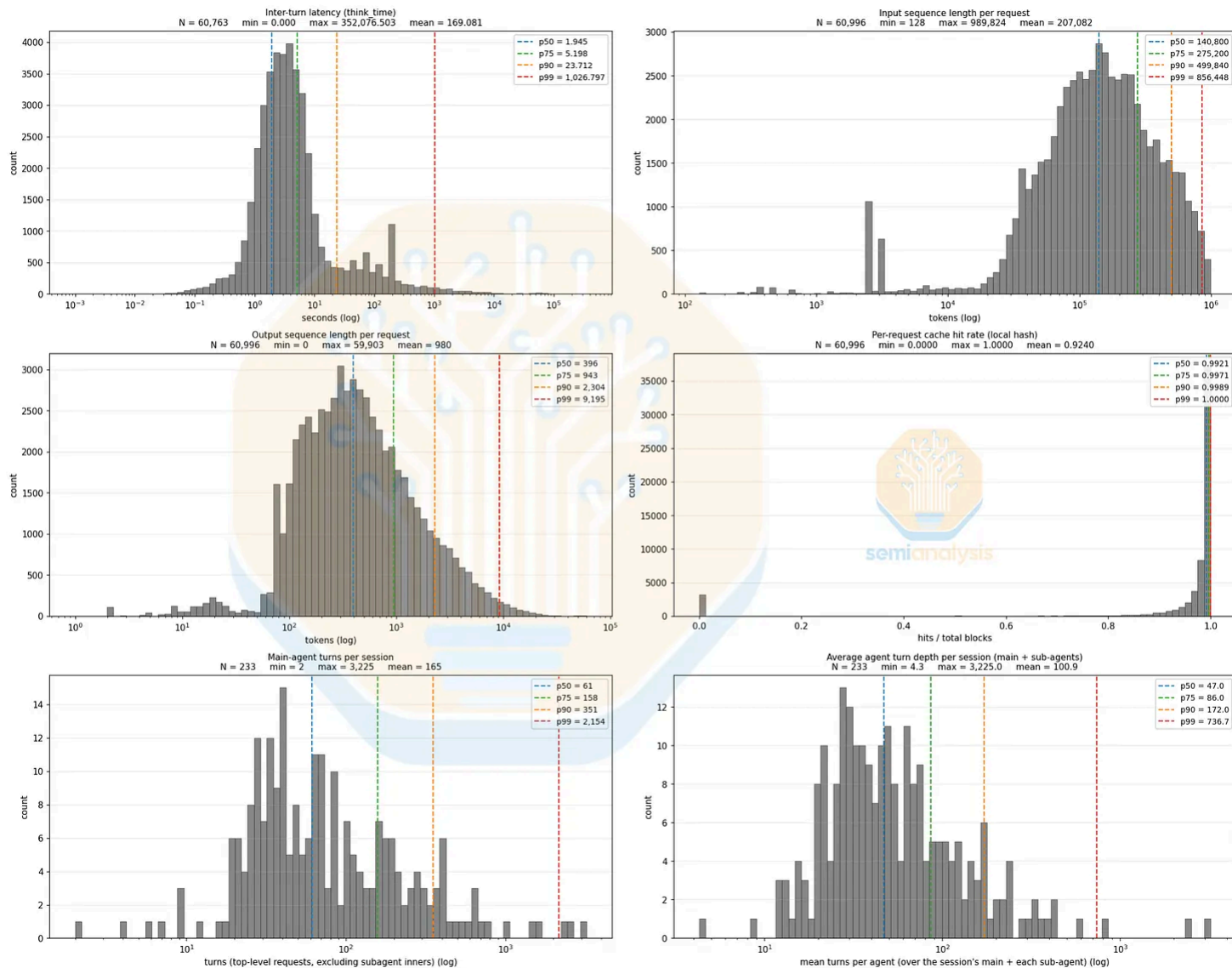
AI 编码代理可以在已知模板内加速调优，但新颖的转换仍需专家判断。单一的持久化内核也降低了传统逐内核分析器时间线的实用性，使得自动化反馈循环更加困难。

Next steps with TileRT<>InferenceX

TileRT<>InferenceX 的后续步骤

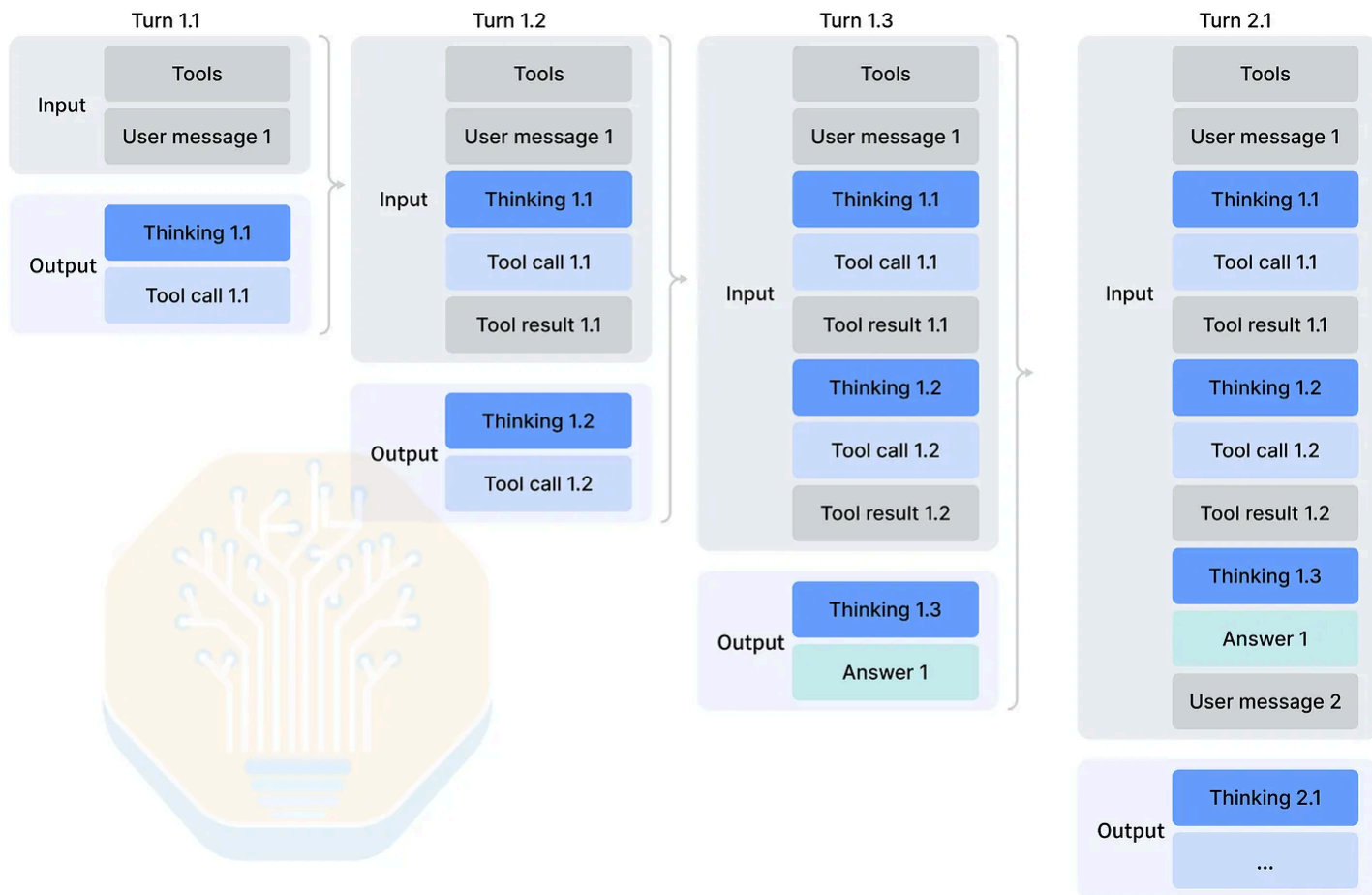
We are actively working on moving TileRT benchmarking from InferenceX's single-turn 8k/1k as well as our new agentic coding benchmark, which we call AgentX. This scenario replays real Claude Code and Codex traces with long-context, multi-turn requests, realistic subagent activity, and dynamic tool-use delays. Its median input length is 140k tokens, while the theoretical median cache-hit rate roofline reaches 99.2%.

我们正积极推进将 TileRT 基准测试从 InferenceX 的单轮 8k/1k 场景，迁移至我们新推出的智能体编程基准测试——我们称之为 AgentX。该场景会回放真实的 Claude Code 和 Codex 操作轨迹，包含长上下文、多轮请求、逼真的子智能体活动以及动态工具调用延迟。其中位输入长度为 14 万 token，而理论中位缓存命中率上限可达 99.2%。

Source: [SemiAnalysis](#) 来源: [SemiAnalysis](#)

This workload will test the entire TileRT<> vLLM system, not just decode speed, including incremental KV transfer, prefix-cache reuse, cache retention and offloading, routing, and scheduling. The critical question is whether TileRT can transfer only the newly introduced context between turns while preserving its ultra-high interactivity advantage.

这一工作负载将全面测试 TileRT 与 vLLM 的协同系统，不仅限于解码速度，还包括增量 KV 传输、前缀缓存复用、缓存保留与卸载、路由及调度等环节。关键在于：TileRT 能否在保持其超高交互性优势的同时，仅传输轮次间新增的上下文内容。



Source: [DeepSeek](#) 来源: [DeepSeek](#)

The second step is to move beyond just batch size one. We will also benchmark TileRT at batch sizes 2, 4, and 8. The goal is to map its throughput–interactivity Pareto frontier and identify the point at which the persistent Engine Kernel’s latency advantage begins to flatten.

第二步是超越仅使用批量大小为 1 的情况。我们还将对 TileRT 在批量大小为 2、4 和 8 时的性能进行基准测试。目标是绘制其吞吐量与交互性的帕累托前沿，并找出持久化引擎内核的延迟优势开始趋于平缓的临界点。

SemiAnalysis is a reader-supported publication. To receive new posts and support our work, consider becoming a subscriber.

SemiAnalysis 是一份由读者支持的刊物。要接收新文章并支持我们的工作，请考虑成为订阅者。

Perf per TCO of TileRT Ultra Fast Speed

TileRT 超快速度的每 TCO 性能

Next, we do an deep dive analysis on TileRT's cost per million output tokens at ultra-high interactivity compared to decode at normal lower-interactivity operating points. The results are quite interesting with TileRT boosting up to 1.9x faster interactivity when iso-cost with traditional engines. [We use our AI TCO Model as baseline for the capex & opex for each chip SKU.](#)

接下来，我们将深入分析 TileRT 在超高交互性场景下每百万输出 token 的成本，并与正常较低交互性工作点的解码成本进行对比。结果相当有趣：在与传统引擎成本相同时，TileRT 的交互速度最高可提升 1.9 倍。我们以 AI TCO 模型作为各芯片 SKU 的资本支出与运营支出基准。

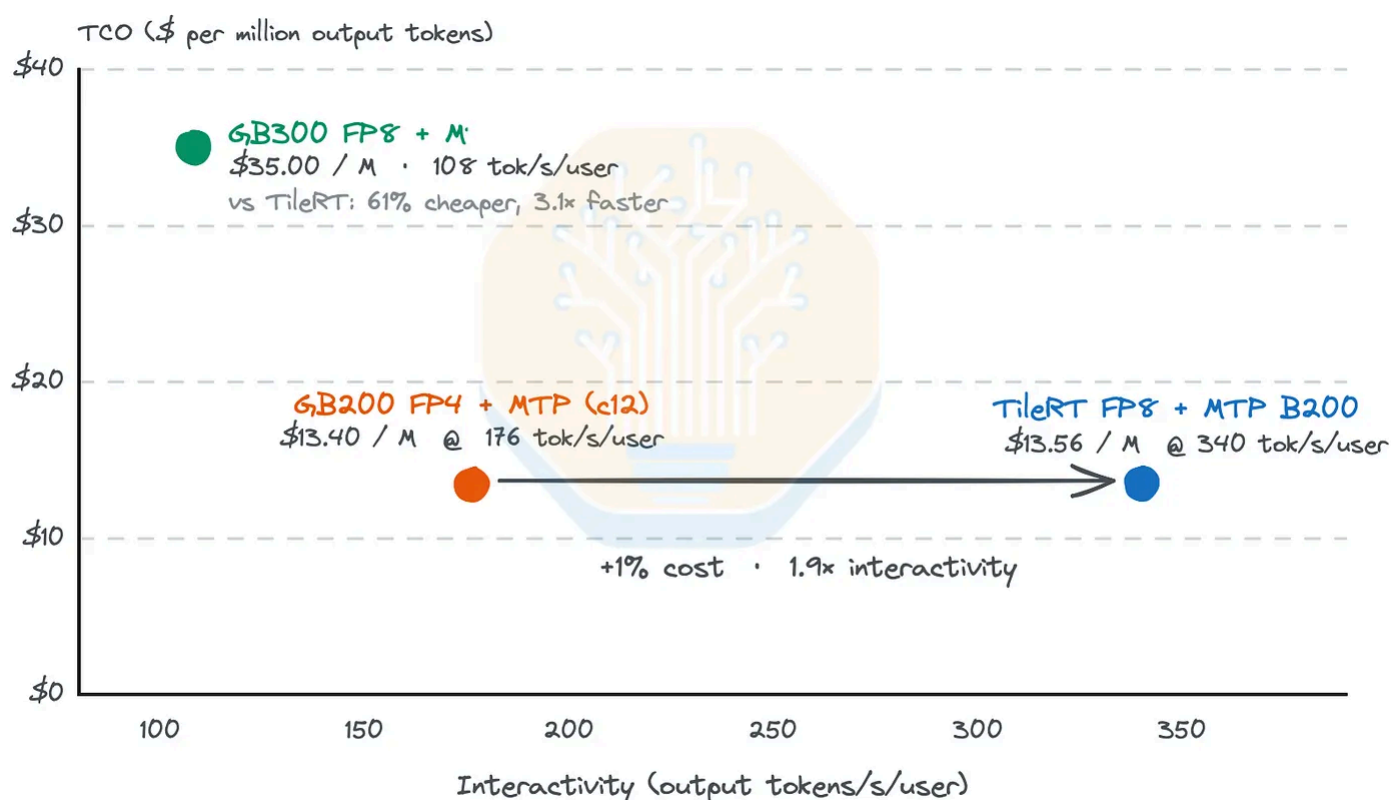
Cost per token is only meaningful after a system satisfies the latency target. At 339 tokens/s/user, the feasible set in our measured GLM-5.1 GPU results only contains TileRT results. The fastest conventional point reaches 176 tokens/s/user, while the fastest conventional disagg FP8 point, GB300 SGLang with MTP reaches only 108.0 tokens/s/user. At 8k/1k, TileRT reaches 340 tokens/s/user while delivering 35.4 output tokens/s per B200. This comes out to \$13.56 per million output tokens.

只有在系统满足延迟目标后，每 token 成本才有意义。在我们实测的 GLM-5.1 GPU 结果中，当用户速率为 339 tokens/s 时，可行方案仅包含 TileRT 的结果。传统方案中最快达到 176 tokens/s/用户，而传统分离式 FP8 方案（GB300 SGLang 结合 MTP）仅达到 108.0 tokens/s/用户。在 8k/1k 配置下，TileRT 实现 340 tokens/s/用户，同时每块 B200 提供 35.4 个输出 token/秒。这相当于每百万输出 token 成本为 13.56 美元。

The FP4 disagg decode setup with the highest interactivity is GB200 FP4 with MTP at concurrency 5. It produces approximately 286 total tokens/s/GPU while maintaining roughly 176 tokens/s/user. At \$1.86 per GB200-hour, that works out to \$13.4 per million output tokens. TileRT therefore costs only 1% more per token while delivering 1.9x the interactivity. This is much more value than Claude code fast mode, up to 2.5x the interactivity but 2x the price per token. The same-precision comparison is even more

striking. The fastest conventional FP8 result is GB300 FP8 with MTP at 108 tokens/s/user. Its endpoint would cost \$35 per million output tokens. TileRT reaches 340 tokens/s/user for \$13.56 per million, 61% cheaper per output token while delivering 3.1× the interactivity.

在最高交互性的 FP4 分离式解码配置中，GB200 FP4 搭配 MTP 在并发数为 5 时表现最佳。该配置每 GPU 每秒可生成约 286 个总 token，同时保持每用户约 176 token/秒的速率。按每 GB200 小时 1.86 美元计算，每百万输出 token 成本为 13.4 美元。因此 TileRT 每 token 成本仅高出 1%，却实现了 1.9 倍的交互性提升。这比 Claude 代码快速模式更具性价比——后者虽能提供 2.5 倍交互性，但每 token 价格翻倍。同精度对比更为显著：传统 FP8 方案中，GB300 FP8 搭配 MTP 最快可达 108 token/秒/用户，其端点成本为每百万输出 token 35 美元。而 TileRT 在实现 340 token/秒/用户的同时，每百万 token 成本仅 13.56 美元，每输出 token 成本降低 61%，交互性提升 3.1 倍。



Source: SemiAnalysis, [TCO Model](#)

SemiAnalysis, TCO 模型

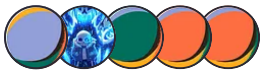


向您的读者推荐 SemiAnalysis

Bridging the gap between the world's most important industry, semiconductors, and business.

连接全球最重要的半导体产业与商业之间的桥梁。

Recommend 推荐



53 Likes 53 个赞 · 1 Restack 1 次重排

← Previous 上一页

Discussion about this post

关于此帖的讨论

Comments 评论 Restacks 转发



Write a comment...